



UNAM · FACULTAD DE CIENCIAS

COMPILADORES

PRÁCTICA III

INTEGRANTES

Gael Emiliano Arreguin Salgado	321121721
Jesus Antonio Romero Godoy	321144292

PROFESORA: Yessica Janeth Pablo Martínez

AYUDANTE: Pedro Yamil Salazar Gonzáles

FECHA DE ENTREGA: 21 de septiembre de 2026

1. Objetivo

Implementar un analizador sintáctico descendente recursivo que determine si un programa cumple la gramática de MiniC. El parser recibe los tokens del lexer de la Práctica 2 mediante una interfaz que reporta errores léxicos y sintáctico y luego se recupera para continuar el análisis

2. Evolución respecto de la Práctica 2

- **Agregados:** `include/parser/parser.h` y `src/parser/parser.c`.
- **Interfaz:** `lexer_scan`, que recorría todo el archivo, se sustituyó por `lexer_init`, `lexer_next_token` y `lexer_destroy`. El estado de lectura (archivo, línea y columna) pasó a la estructura `Lexer`
- **Sin impresión:** las funciones que imprimían cada token ahora lo construyen y lo entregan; `lexer.c` o sea ya no escribe en ninguna salida.
- `main.c` ahora coordina lexer y parser, y las pruebas de las Prácticas 1 y 2 se movieron a `tests/lexer/`

3. Nuestra nueva interfaz

- `main.c` aquí valida argumentos, abre y cierra el archivo, inicializa lexer y parser, muestra el resultado y devuelve el código de salida
- **Lexer** entrega un token por llamada. `LexerStatus` distingue un resultado normal (incluido un token `ERROR`) de un fallo de lectura o de memoria
- **Parser:** su estado guarda el token (`current`), el último consumido (`previous`) y las banderas `had_error`, `panic_mode` e `internal_failure`. Solo muestra `parser_init`, `parser_parse_program` y `parser_destroy`
- **Fallo** aquí el parser lo reporta, activa `internal_failure` y reemplaza `current` por un `TOKEN_EOF` sin lexema, con lo que todas las producciones terminan de forma natural.

4. Gramática

Cada no terminal tiene su propia función, que devuelve 1 si reconoció la producción y 0 al encontrar un error. `parse_statement` elige la producción con el token de `current` `INT` o `BOOL` inician una declaración, `IDENTIFIER` una asignación, y `PRINT`, `IF`, `WHILE` y `LBRACE` sus sentencias `parse_statement_list` implementa `statement*` tanto para el programa (hasta `TOKEN_EOF`) como para los bloques (hasta `}`), por lo que un archivo vacío es válido. El `else` se busca justo después de la rama verdadera, así que queda asociado al `if` más cercano

5. Expresiones

Hay una función por cada nivel de precedencia, desde `parse_expression` hasta `parse_primary`. Cada nivel obtiene sus operandos del siguiente de mayor precedencia, por lo que en $5 + 2 * 3$ el producto se reconoce antes que la suma. Los operadores binarios usan un ciclo (`op operando`)*

que justo da asociatividad a la izquierda; `parse_unary_expression` se llama a sí misma después del `-` lo que da asociatividad derecha

6. Resultados y pruebas

Conjunto	Válidos	Inválidos	Total
tests/public/	8	9	17
tests/propias/	21	11	32

Entrada	Diagnóstico
<code>int x = 5 / print(x);</code>	<code>[2:0]</code> se esperaba <code>';</code> ... <code>'print'</code>
<code>\$</code> entre dos sentencias	solo Error lexico <code>[2:0]</code>
<code>if (a)</code>	<code>[1:6]</code> ... se encontro <code>TOKEN_EOF</code>

Las pruebas de las Prácticas 1 y 2 se conservan en `tests/lexer/`, pero ya no se ejecutan con `minic` porque este no imprime tokens

7. Problemas encontrados

- En casos como `if (a) }` el mismo token podía reportarse dos veces. LO resolvimos con `panic_mode`.
- Las pruebas léxicas comparaban la salida de `minic` que ya no imprime tokens. Estos se movieron a `tests/lexer/` para conservarlas.

8. División del trabajo

Ambos integrantes revisamos la implementación completa y podemos explicar,

En esta ocasión ambos nos centramos en los tests y en que el código queara legible, suamos todo tipo de documentación y herramienta para que el código se vea legible, haya buenas prácticas y sea correcto, lo verificamos con las pruebas, con nuestro diseño (del que recibimos ayuda) y prueba y error. AMbos hicimos de todo, incluso al documentar cambiabamos y nos apoyabamos de herramientas para una correcta redacción.

9. Uso de herramientas de Inteligencia Artificial

FALTA poner nuestros prompts de redacción, refactorización y fotos de la ayudda en comprender el script y en pasar los tests.

10. Conclusiones

Fue una práctica que nos costó al final, pero fue muy buena para entender el parser. Seguimos la gramática y las instrucciones y nos quedamos con una buena versión final.

Relacionamos cada producción con una función nos ayuda a hacer que la estructura del parser siga directamente a la gramática, incluidas la precedencia y la asociatividad. La parte más

importante fue garantizar que cada error consuma un token o devuelva el control a quien puede procesarlo

11. Referencias

- Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2007). *Compilers: Principles, Techniques, and Tools* (2.^a ed.). Pearson Addison-Wesley. Capítulo 4 (secciones 4.1.4 y 4.4.1).
- Pablo Martínez, Y. J. y Salazar González, P. Y. (2026). *Compiladores — Práctica 3 y material complementario*. Facultad de Ciencias, UNAM.
- Pablo Martínez, Y. J. y Salazar González, P. Y. (2026). *Definición de MiniC*. Facultad de Ciencias, UNAM.